

BYOF Agent — Technical Document

Production Planning · Team LIZZ · BYOF Agent v1.0.0 · 27 September 2026

BYOF Agent helps a production manager respond to disruptions in a discrete-assembly plant. When an order changes, a delivery slips, a machine stops or an operator is absent, one Agent investigates the factory facts, asks deterministic services for checked and priced response options, recommends one, and — after the manager approves it on an option card — carries it out through controlled business commands until the factory accepts the new plan. This document explains how the system works: what the language model decides, what the services enforce, how approval turns into factory changes, and how the results were verified. Source references are relative paths in the public repository github.com/narmi924/byof-agent.

1. Scope and requirements

The target user is the production manager of a small or mid-sized plant with bills of materials, routings, shared workstations and operator skills. The product has two browser identities: the **manager**, who works in one conversation per disruption, and the **disruption simulator**, which plays the shop floor in the demonstration by creating twelve kinds of disruption in a simulated factory (order changes and new orders; late, short or cancelled receipts; stock count losses; machine stops and breakdowns; temporary leave and absence; failed quality checks; random breakdowns).

ID	Functional requirement	Where it is met
F1	Read orders, stock, receipts, resources, staff and work in progress from the factory and keep a versioned snapshot	Section 3
F2	For a disruption, build response options from concrete measures, then schedule, check and price each	Section 5
F3	Explain the situation in plain language, recommend one option, answer follow-up questions	Section 4
F4	Start execution only from a manager approval on a concrete option	Section 6
F5	Apply the measures, reschedule within the approved scope, release the plan and record the factory's acceptance	Section 6
F6	Continue waiting work when people confirm missing facts; report execution results in the conversation	Sections 4, 6

ID	Non-functional requirement	Mechanism
N1	The model cannot approve or change the factory directly	Strict action contract; execution approval only from a manager-role demo session with a CSRF token
N2	No schedule is released unless it is valid	Independent Checker on every candidate
N3	Execution is idempotent and resumable	Execution ledger with stable request IDs
N4	Every decision is traceable	Conversations, jobs, candidates, ledger and receipts stored in PostgreSQL
N5	Bounded model use and search time	≤ 8 model requests per turn, ≤ 40 per conversation; option search ≤ 120 s
N6	Unknown facts are confirmed by people, never estimated by the model	Solver refusal plus confirmation tasks

Out of scope in this release: a real ERP or MES connection, real purchase orders, contracting with customers, email notifications (the code exists but is disabled), several plants, and user management beyond two demonstration roles.

2. Architecture

Component	Technology and entry point	Responsibility
Web	React and TypeScript (Vite), served by NGINX	Manager workbench and simulator; proxies <code>/api</code>

Component	Technology and entry point	Responsibility
API	FastAPI, <code>services/api/main.py</code>	Sessions, roles and CSRF; routers for planning, assistant, cases, notifications, models and preferences; approvals
Business worker	<code>services/worker/main.py</code>	Fact synchronisation, the case runtime (the Agent), the execution ledger and plan release
Solver worker	<code>services/solver_worker/main.py</code>	CP-SAT solve jobs and option comparisons, each checked and priced
Factory service and clock	<code>services/factory_sim/</code>	Simulated factory with its own database, HTTP API and running clock
Database	PostgreSQL 18, SQLAlchemy and Alembic	Conversations and checkpoints, snapshots, jobs, candidates, releases, ledger
Model providers	<code>packages/providers/catalog.py</code>	DeepSeek Flash (default) or Claude Sonnet 4.5 through a hosted gateway, chosen per account

Figure 1 (next page) shows the components and the boundaries between them.

Trust boundaries.

- *Model*. The model receives business context as data and returns text. The server parses it into one registered action, rejects anything else and runs the tool itself. The model has no credentials and no SQL, shell, file or network access; each turn uses one provider with no silent fallback, and keys stay in the server environment.
- *Human*. Execution approval exists only as a request from a manager-role demo session on an option card, carrying the `HttpOnly` session cookie and a CSRF token derived from the session with HMAC (`packages/auth.py`). The current release has two demonstration roles without passwords; real sign-in is a pilot requirement (section 10). Chat text never counts as approval.
- *Factory*. BYOF reads and changes the factory only over HTTP through a connector with declared field mappings (`packages/integrations/mapped_http.py`), using separate credentials for reading, executing approved measures and simulator control. The simulated factory keeps its data under its own database role. In a pilot, a plant adapter would take the simulator's place behind the same HTTP contract; no plant adapter exists yet.

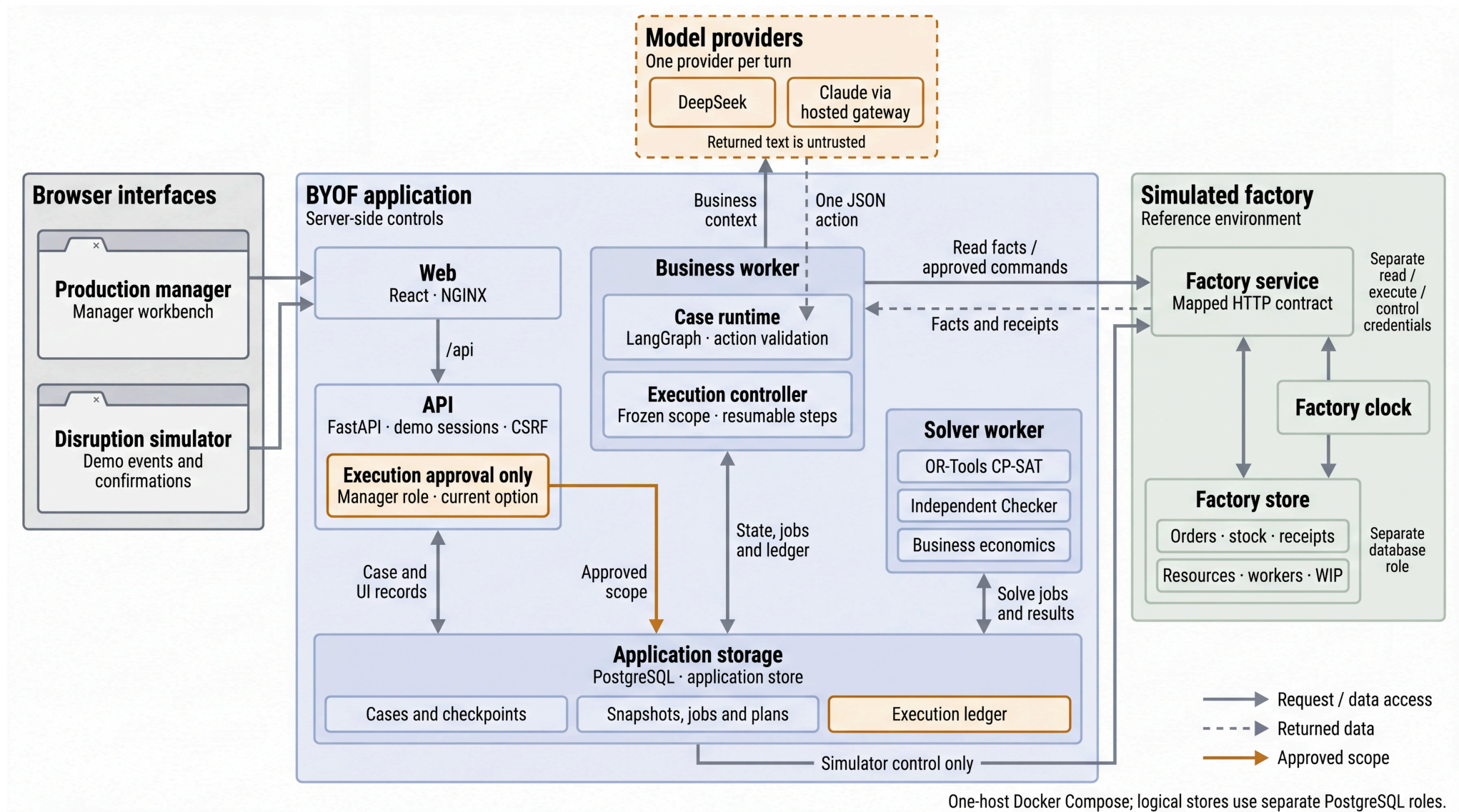


Figure 1. System architecture and trust boundaries. The business worker sends business context to one selected model provider per turn, with no silent fallback, and treats the returned JSON action as untrusted input. The orange path is execution approval only; investigation and fact reads do not wait for it. BYOF reaches the simulated factory only through its mapped HTTP contract with separate read, execute and control credentials; in a pilot, a plant adapter would replace the simulator behind the same contract (section 10).

3. Domain data and modelling

The reference factory models the final assembly of four sealed deep groove ball bearings, 6202–6205-2RS1. Product data and general practice follow public SKF information; the plant layout, operation times, orders, stock, calendars and prices are synthetic. SKF is not involved and the scenario does not describe an SKF plant.

Layer	Content	Origin	Where
1. Public knowledge	Dimensions and catalog weights; structure (rings, ball set, cage, two RS1 seals, grease fill); general assembly and quality practice	SKF public product pages and catalogs	<code>docs/DOMAIN.md</code> (links)
2. Planning abstraction	Eight-operation routing OP10–OP80; eight workstations and five skills; seven-component bill of materials (inner ring, outer ring, ball set, cage, two seals, grease, box), each line naming the operation that consumes it; batches of 50; changeovers of 0, 1 or 5 min; 60-minute freeze window	Team design based on layer 1	<code>database/seed/</code>
3. Synthetic operations	Six orders (5,400 pcs, 108 batches, 864 operations) due over five days; opening stock and 16 confirmed receipts; eight workers with shifts and an optional overtime window; price catalog	Synthetic, declared as such	<code>database/seed/</code> , <code>packages/domain/economics.py</code>
4. Validation	Database keys and checks; data validation script (counts, BOM and routing completeness, qualified resources, batches, durations, calendars); SHA-256 pinned baseline that is refused if changed	Implementation	<code>database/scripts/validate_data.py</code> , <code>data/development/skf-baseline-hashes.json</code>

Snapshot contract. Every calculation works on a `Snapshot` (`packages/domain/models.py`): factory and run identifiers, snapshot clock, planning horizon, source envelope, planning revision, active plan and its hash, factory profile (routing, bill of materials, policy), orders, inventory, receipts, resources, workers, actual execution, reservations, production batches, business terms and a content hash. A solve that references an out-of-date snapshot or source is refused rather than guessed.

Pilot data. A plant maps its own exports — orders, bills of materials, routings with times, workstations and skills, stock and receipts, calendars and cost assumptions — to the same contract through the connector's declared field mappings, and the validation layer runs before anything is planned. No plant data has been mapped yet. Figure 2 (next page) traces where the data and each number come from.

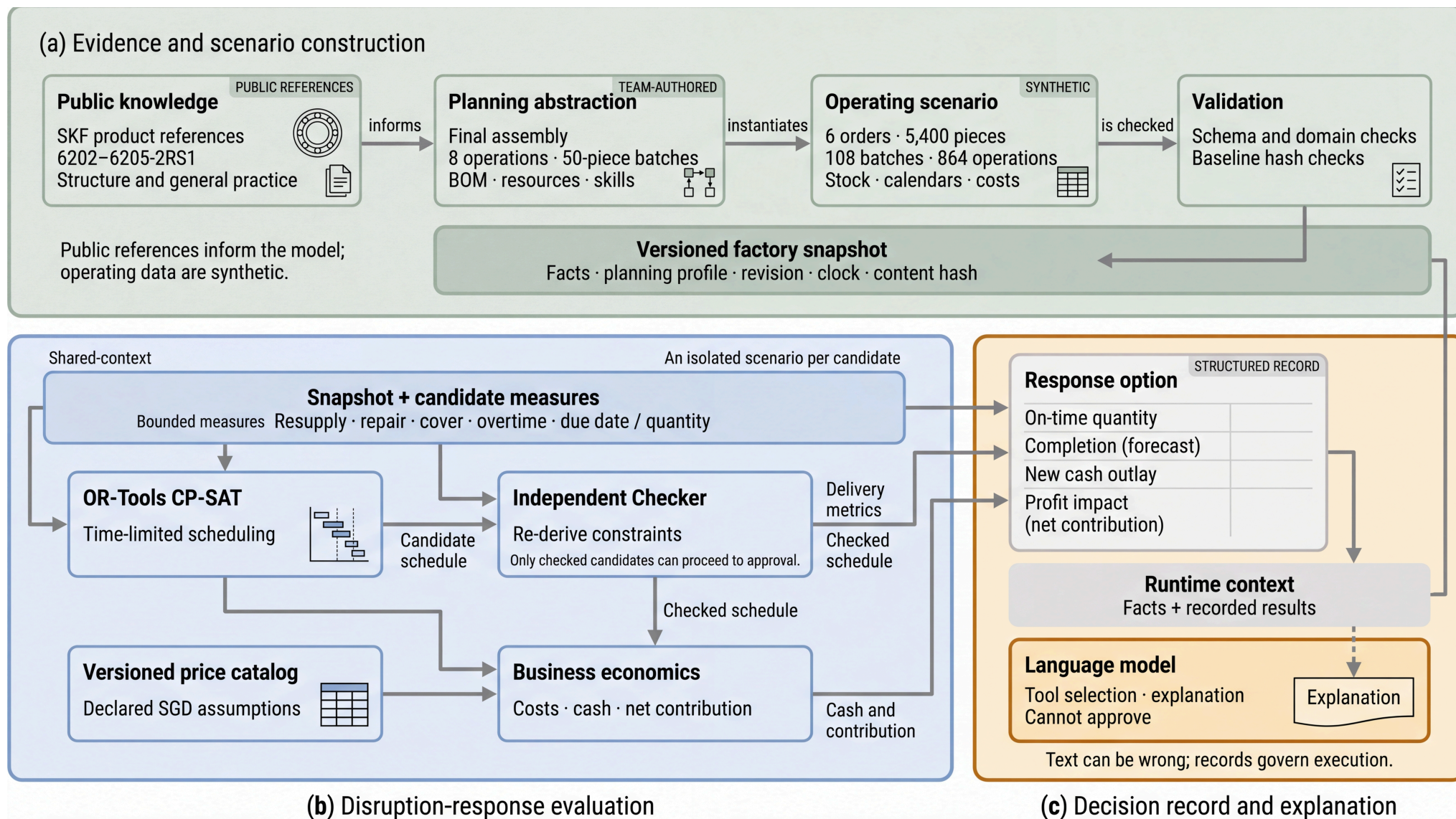


Figure 2. Data and model provenance. (a) Public product references inform the team's planning abstraction; the operating scenario is synthetic and validated before it becomes a versioned factory snapshot. (b) For a disruption, the snapshot and a bounded set of candidate measures are scheduled with CP-SAT and re-checked by the independent Checker; business economics prices each checked schedule from the versioned price catalog. (c) The structured response option holds the metrics; the language model receives facts and recorded results through the runtime and writes the explanation. Links to the model are logical information dependencies mediated by the runtime, not direct provider access to the database. Model text can be wrong; approval and execution use the structured record.

4. Agent flow

Each disruption is one conversation, run by a LangGraph graph with a PostgreSQL checkpoint

(`packages/agent/case_runtime.py`, `_graph`): **refresh** re-reads the factory facts; **decide** sends one model request and parses one action; **execute** runs the registered tool on the server. From decide the graph goes to execute, back to refresh or to **wait**; from execute it ends, waits or refreshes. Wait is a LangGraph interrupt: the conversation is saved and costs nothing until something wakes it.

Action contract (`packages/agent/decisions.py`, `Action`). The model must return exactly one JSON object `{action, parameters, reason_summary}` of at most 8,192 characters, validated against a discriminated union of 13 actions:

Purpose	Actions
Read facts	<code>query</code> (orders, stock, receipts, resources, workers, actuals, policy, terms), <code>report_production</code> (delivery facts computed from the plan in effect)
Compute	<code>solve_scenario</code> , <code>evaluate_business_options</code> , <code>compare_candidates</code>
Involve people	<code>request_information</code> (ask a role for up to five facts by a deadline), <code>handoff</code> , <code>request_approval</code> , <code>propose_preference</code> , <code>propose_simulation</code>
Converse and control	<code>reply</code> , <code>wait</code> , <code>finish</code> (requires an existing release ID as evidence)

A response that fails the contract runs no tool. The next request receives only the schema paths and error kinds of the failure — not the rejected text — and one correction is allowed; a second failure stops the turn. Business context is framed as data, not instructions, and the prompt is capped at 120,000 characters.

Budgets. At most 8 model requests per turn and 40 per conversation, a 120-second deadline per turn and 2 solves per turn. Without new business input, a waiting conversation rechecks itself at most 3 times and then waits for a person. An automatic wake-up does not refresh the budget; only a new manager message can change the search conditions.

What starts a turn. A manager message; opening a field change (the business worker suggests one when synchronised facts differ); a completed solve or comparison job (`wake_completed_jobs`); a shop-floor confirmation of remaining work that a waiting conversation needs (`wake_for_confirmed_work`, `packages/agent/cases.py`); and the recheck timer of a `wait` action. After an approved execution finishes, the first reply is written from the recorded execution result, not by the model. Figure 3 (next page) follows one disruption from investigation through the manager's approval to the factory's acceptance.

From disruption to factory acceptance

BYOF Agent | One Agent, deterministic services and an explicit approval boundary

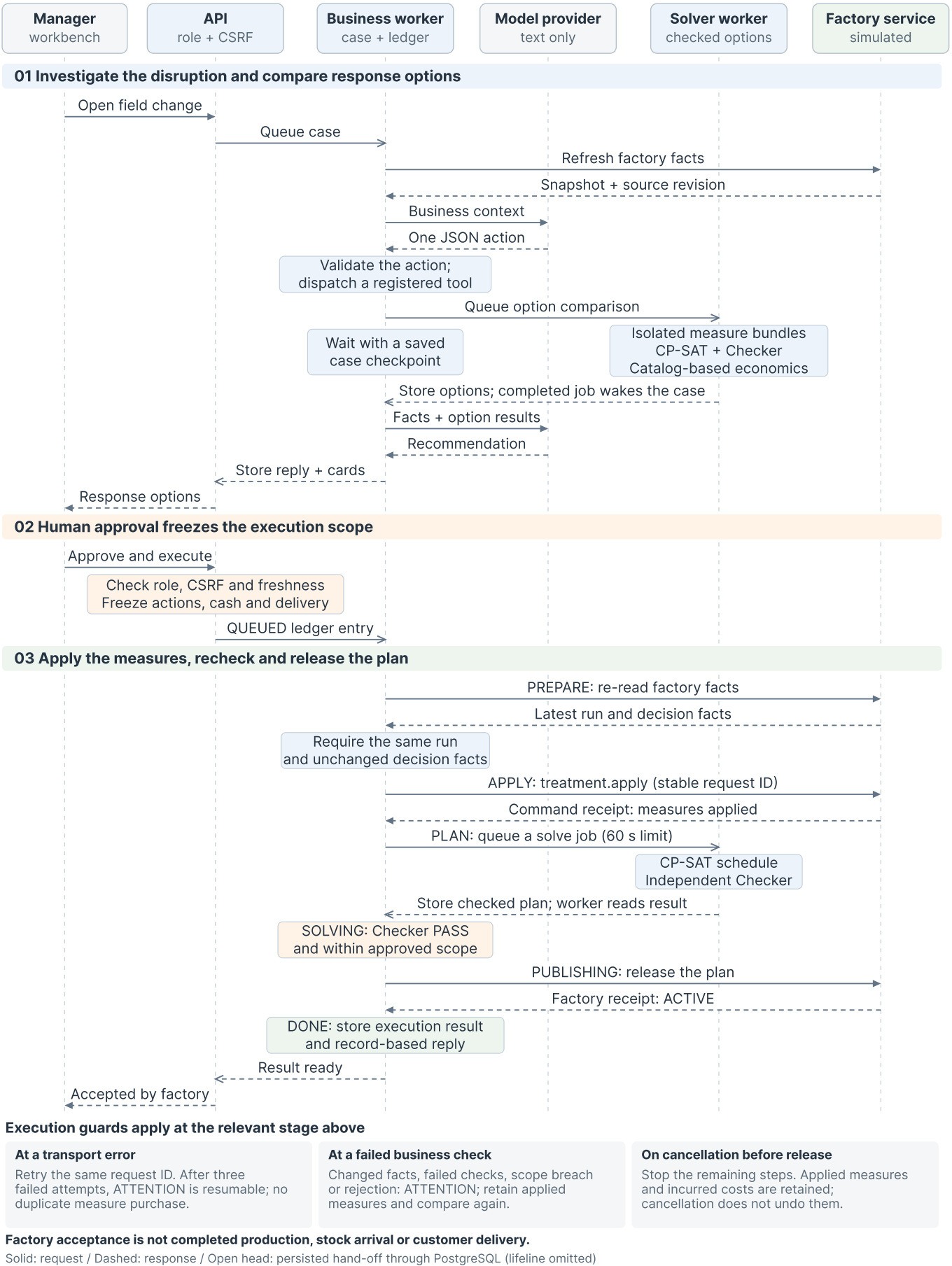


Figure 3. One disruption from investigation to factory acceptance: (01) the Agent investigates and the solver worker compares checked options; (02) the manager's approval freezes the execution scope; (03) the execution ledger applies the measures, re-plans and releases. Open arrow heads are hand-offs persisted in PostgreSQL; the guards at the bottom apply at the stage where they occur, not after DONE. Factory acceptance is not completed production, stock arrival or delivery.

5. Scheduling and economics

CP-SAT model (`packages/planning/solver.py`). Each operation has a start, an end, a machine choice and a worker choice. Constraints: routing precedence inside each batch; no overlap on a machine, with changeovers of 0, 1 or 5 minutes; no overlap for a worker, who must hold the skill; shift calendars, with overtime windows usable only when allowed; materials as a full kit per batch — every component of an unstarted batch must be available (opening stock minus reservations plus *confirmed* receipts that have arrived) when the batch starts its first operation, a rule the Checker applies in the same way, while each BOM line's consuming operation is used to reconcile actual consumption during execution; work already started is kept, and operations that start inside the 60-minute freeze window stay in place.

Objectives. Five objectives in lexicographic order, `OBJECTIVES`: weighted tardiness ($\sum \text{priority weight} \times \text{minutes late}$), incremental overtime, changed operations against the active plan, total start shift, makespan. The manager can choose `delivery_first` (this order), `stability_first` or `overtime_first` (`OBJECTIVE_ORDERS`, `packages/agent/case_tools.py`). The search minimises one level at a time; a level proven optimal is fixed and the next is searched with the previous solution as a hint. If a level is only feasible when time runs out, the search stops and keeps that schedule. Each level also stops once a feasible solution has not improved for 4 seconds; a fixed seed (17) and one search worker are used to improve repeatability, and reported outcomes are tied to the recorded environment and run. **Feasible does not mean optimal, and UNKNOWN (nothing found in time) does not mean infeasible.**

Refusals. The solver refuses to plan when the remaining work of an interrupted operation is unknown (`WIP_CONFIRMATION_REQUIRED`) or when the source is incomplete or stale; the gap goes to a person instead.

Plan today. The first plan of the day without disruptions comes from a preset keyed by the opening snapshot's fingerprint (`packages/planning/presets.py`) and is verified by the Checker when it is used, so its few seconds are not a from-scratch solving time. Disruption-response schedules are solved online and checked independently.

Independent Checker (`packages/planning/checker.py`, `check_candidate`). It re-derives every rule from the snapshot, without the solver's model, and reports issue codes such as `MISSING_OPERATION`, `PRECEDENCE`, `MATERIAL_SHORTAGE`, `WORKER_QUALIFICATION`, `RESOURCE_UNAVAILABLE`, `FROZEN_OPERATION_CHANGED`, `HARD_DEADLINE`, `STALE_TIME` or `OBJECTIVE_MISMATCH`. Only a candidate with status `PASS` can be approved or released.

Response options (`packages/planning/business_options.py`, `packages/domain/treatment.py`). The service compares a bounded set of supported measure bundles, built from stock, qualifications and the catalog: standard, expedited or immediate resupply; expedited repair; qualified cover; overtime; a later due date; a smaller quantity. Each bundle is applied to an isolated copy of the snapshot, scheduled, checked and priced. Other unfinished orders keep their promised due dates as hard constraints. When no measure keeps a due date, options propose a new date for that order, which the manager must confirm as agreed with the customer. The comparison shares one budget of at most 120 seconds; each bundle runs in its own process for at most 40 seconds, in parallel across the available CPUs. This is not an exhaustive search over all possible interventions.

Economics (`packages/planning/business_economics.py`, catalog `byof-demo-economics/1`). For each feasible option, with q the quantity, p the price and v the variable cost of each order:

Quantity	Definition
Revenue R	$\sum q \cdot p$
Variable cost V	$\sum q \cdot v$ — includes regular material and labour, so extra purchases add only premiums and freight
Late deduction L	Per order 1 % of its sales per day late, pro-rated by minute, at most 20 %
Additional cost A	Overtime premium for overtime not yet worked + expedite surcharges + catalog handling costs of the measures
Net contribution	$N = R - V - A - L$, summed over all open orders in the option's snapshot (shown as "profit impact"; an absolute projection, not accounting profit and not an increase)
New cash outlay	Only the extra money the option spends

Quantity	Definition
Adverse case	Surcharges and overtime premiums +20 % and every open order 60 minutes later; costs recalculated, not rescheduled; not a probability

Example (section 9, standard resupply for SO-003): across the six open orders due within the planning horizon, $R = \text{SGD } 87,000$, $V = \text{SGD } 58,500$, $A = \text{SGD } 50$ (transport of five resupply lines) and $L = 0$, so $N = \text{SGD } 28,450$. The SGD 1,200 of new cash pays for the extra material, whose regular cost is already part of V .

Catalog assumptions (SGD): product price / variable cost per piece 6202 12 / 8, 6203 14 / 9.50, 6204 18 / 12, 6205 22 / 15; standard resupply 120 min lead time plus SGD 10 transport; expedited 30 min, +25 %, SGD 30; immediate transfer at once, +100 %, SGD 80; expedited repair SGD 500 (60 min); qualified cover SGD 200 (30 min); overtime premium SGD 18 per hour.

Ranking. Options above the manager's cash limit are blocked. Among the rest, an option is dominated if another is at least as good on six metrics — net contribution, new cash, weighted lateness, changed operations, added overtime and pieces not delivered — and better on one. Non-dominated options come first, ordered by the manager's current goal (net contribution by default; cash, delivery, stability or overtime on request). This is a comparison of the options found, not a claim of a global economic optimum.

6. Human approval and controlled execution

Approval. "Approve and execute" on an option card is accepted only from a manager-role demo session with a valid CSRF token and only while the option is still current. It freezes the scope: the listed measures, the new cash outlay and the delivery impact. Options with overtime or a changed customer order need the manager to confirm overtime and customer agreement on the same card. The approval creates one entry in the execution ledger (`packages/agent/treatment_execution.py`, `approve_treatment`). Phases 02 and 03 of Figure 3 show these steps in context.

Execution ledger. A business worker advances each entry one stage at a time and stores the result after every step:

Stage	What happens	Stops with
PREPARE	Re-read the facts; the factory run and the business facts that the decision depends on must match the approval	<code>SOURCE_RUN_CHANGED</code> , <code>STUDY_FACTS_CHANGED</code>
APPLY	Send <code>treatment.apply</code> with request ID <code>treatment:<ledger entry ID></code> , the expected snapshot hash and the catalog version; if only the clock moved, resend against the latest facts (at most 3 times)	<code>STUDY_FACTS_CHANGED</code> if the decision facts changed meanwhile
PLAN	Queue a solve job (60 s) with request ID <code>treatment-plan:<ledger entry ID></code>	—
SOLVING	Require a solution with Checker PASS that stays within the approved scope; record the release under the manager's approval	<code>PLAN_NOT_VERIFIED</code> , scope exceeded
PUBLISHING	Wait for the factory's receipt of the release: ACTIVE or REJECTED	<code>SOURCE_REJECTED</code>
DONE	Execution result stored; production and receipts follow the plan	—

The entry's state is QUEUED while stages advance, then DONE, ATTENTION or CANCELLED. A connector error (lost or late answer) retries the same request, so nothing is bought twice; after three attempts the entry needs attention but can be resumed. Changed facts, an unverified plan, a plan outside the approved scope or a rejected release also need attention and cannot be resumed: completed measures are kept and the manager compares again.

Three different "undo" requests.

- *Stopping an analysis* ends automatic continuation of a conversation. It changes no factory fact and does not touch approved executions.
- *Cancelling an execution* stops its remaining steps. It is allowed until the plan is released (after that: `PLAN_ALREADY_DISPATCHED`); during APPLY the factory is asked to cancel the treatment. Measures already applied and costs already incurred stay; nothing is refunded.
- *Undoing completed operations* does not exist. Reversing an applied measure is a new decision with its own options and approval.

Facts only people know. When an interrupted operation's remaining work is unknown, the solver refuses to plan and the conversation waits. The shop floor confirms the remaining work on screen; `wake_for_confirmed_work` then continues every conversation that was waiting for it.

7. User interface and traceability

The manager's workbench has a conversation list, the conversation with the Agent's replies and option cards, a production timeline, an execution log and settings (model and recommendation preferences). The simulator has the factory facts in four groups, disruption commands and confirmation tasks. Figure 4 shows an option card and the execution log from a run of the same scenario in the English build.

For one decision, the following records are stored and linked; the workbench shows the conversation, the cards, the execution log and the timeline.

Step	Record
Disruption	Simulator command and its receipt; the resulting field change
Investigation	Conversation messages and tool activity with the snapshot each tool used
Options	Option cards with candidate, Checker result, catalog version and assumptions
Approval	Approving user, time and frozen scope
Execution	Ledger stages, factory command receipts, release ID and factory receipt
Follow-up	Reply from the recorded result; production and receipts on the timeline

● Agent

SO-003 demand rose from 1200 to 1500 pcs. The verified material gap is 250 EA each of Inner Ring, Outer Ring, Ball Set and Cage, plus 500 EA of RS1 Seal, so the current schedule cannot cover all orders. Four checked options are on the cards: Standard resupply (all 1500 on time, about SGD 28,450 net contribution, SGD 1,200 new cash); Negotiate a smaller quantity of 1200 with a new due date of 29 Sep 16:00 (about SGD 26,506 net contribution, no new cash, 1200 late); Expedited resupply (all 1500 on time, about SGD 28,063, SGD 1,588 new cash); Immediate transfer from local emergency stock (all 1500 on time, about SGD 26,950, SGD 2,700 new cash). I recommend Standard resupply: it keeps all 1500 on time at the lowest extra cash. Approve on the card to proceed.

Response options

2 available · ranked by profit impact · shop floor at 2026-09-27 08:30

Analyzing order SO-003; the options below may propose a new due date for this order.

Option 1

● Recommended

Standard resupply and required resource measures

On time	Completion	New cash outlay	Profit impact	Added overtime
1500 / 1500 pcs	2026-09-28 14:58	SGD 1,200.00	SGD 28,450.00	0 staff-min

Changed operations

386 ops

Resupply IR-6204 · 250 EA · 2026-09-27 10:45

Resupply OR-6204 · 250 EA · 2026-09-27 10:45

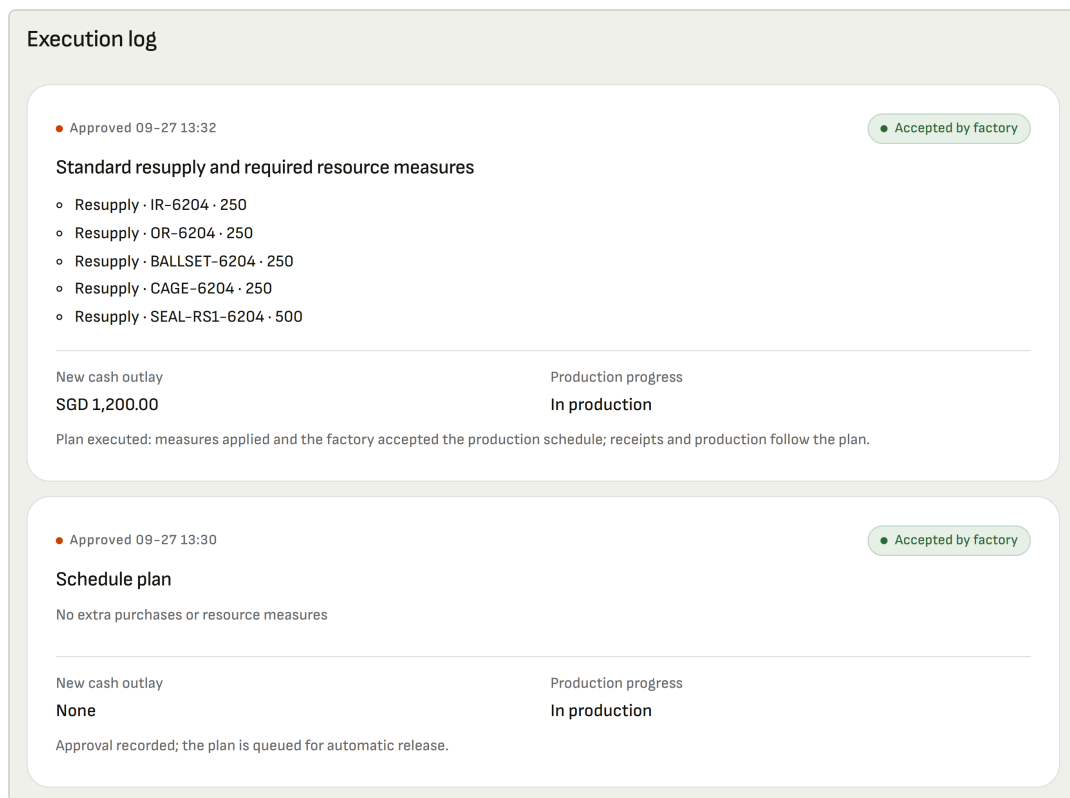
Resupply BALLSET-6204 · 250 SET · 2026-09-27 10:45

Resupply CAGE-6204 · 250 SET · 2026-09-27 10:45

Resupply SEAL-RS1-6204 · 500 EA · 2026-09-27 10:45

1. Full delivery: 1500 pcs, 2026-09-28 17:30

(a) Agent reply and the recommended option card



(b) Execution log after that option was approved

Figure 4. Interface evidence from the English build (2026-09-27). (a) After SO-003 rose from 1,200 to 1,500 pieces: 1,500 of 1,500 pieces on time, new cash outlay SGD 1,200.00, profit impact SGD 28,450.00 (projected net contribution across the six open orders). The Agent's free text is shown as recorded; the card metrics are the authoritative values. (b) The approved standard resupply (five measures, new cash SGD 1,200.00) and the day plan, both accepted by the factory. The approval times are the computer's clock, not factory time.

8. Deployment

Current design (used for all runs in this document). Docker Compose on one machine (`compose.yaml`). Seven long-running services — PostgreSQL, factory service, API, solver worker, business worker, factory clock and web — plus one-shot jobs that create database roles, run migrations and load the reference factory. Only the web service publishes a port, bound by default to `127.0.0.1:18080`; every service except the factory clock has a health check. Secrets (database roles, factory credentials, model keys) live in a private environment file outside the repository, and PostgreSQL data is kept in a named volume that ordinary restarts do not clear. Setup is described in `docs/GUIDE.md`.

Pilot deployment (design, not deployed). One small server running the same Compose project behind an HTTPS reverse proxy, with the web port still on loopback; scheduled PostgreSQL backups; model keys in the server environment; real sign-in instead of the two demonstration roles; a plant adapter in place of the simulator. At the time of writing no public deployment has been verified, and no performance figure in this document comes from a server.

9. Verification

Each row is a separate observation. Run times are single measurements on a laptop, not averages; subsets from earlier builds are not added to later totals.

ID	Build and environment	Model · Agent	Scenario	Runs	Result
V1	1.9.5 (same code as v1.0.0); GitHub Actions; 2026-09-27	none · no	Backend: lint, types, full test suite including a continuous-factory case. Frontend: lint, tests, build. Containers: build, start, setup, HTTP smoke test	1 per job	All three jobs passed

ID	Build and environment	Model - Agent	Scenario	Runs	Result
V2	Public v1.0.0 tree; local, isolated PostgreSQL; 2026-09-27	none · no	Frontend check (297 tests); backend full suite	1	Frontend passed. Backend 1,868 passed, 1 failed: a test fixture still carried an old data digest. Fixture corrected; the two affected contract test files re-run: 38 passed
V3	English build 1.9.5; local Docker; 2026-09-27	DeepSeek · Flash · yes	Plan today → approve → SO-003 1,200 → 1,500 pcs → options	1	Options with a recommendation 43 s after the change was opened; 8 model requests
V4	1.9.0, Chinese interface; local Docker; 2026-09-26	DeepSeek · yes	SO-003 +300 pcs, approve standard resupply, follow-up question, temporary leave	1	Options about 30 s; execution 33 s; leave rescheduled in 29 s and accepted 10 s after approval
V5	1.9.x before localization; local Docker; 2026-09-26/27	DeepSeek · yes	SO-002 1,000 pcs due the same day 12:00; busy worker absent	1 each	Three options with new due dates in about 50 s, executed in about 45 s; cover option about 1 min after remaining work was confirmed, executed in about 40 s
V6	1.9.0 services; 2026-09-26	none · no	Option comparison without the model for 12 disruption kinds	1 per kind	Feasible options in 15–37 s each; not repeated on 1.9.5
V7	English build 1.9.5; local Docker; own database; 2026-09-27	DeepSeek · Flash · yes	As V3, then approve standard resupply	1	Day plan 4 s after the request (preset), accepted 1 s after approval; options and recommendation 47 s after the change was opened; factory accepted the new plan 25 s after approval; 11 model requests in two conversations
V8	English build 1.9.5; local Docker; own database; 2026-09-27	DeepSeek · Flash · yes	As V7, then SO-002 800 → 1,000 pcs due the same day 12:00; a question on one option; approve the full-quantity option with immediate stock and overtime	1	SO-003 options about 44 s, executed 28 s after approval. SO-002: three options with new due dates in 42 s — next day 10:00 (recommended), 800 pcs by 18:00, all 1,000 pcs by 19:00 with 562 staff-min of overtime; that option executed 28 s after approval, SO-002 forecast 18:03 against the agreed 19:00; 19 counted model requests

Option results stored in V3 (same values in V7 and V8): standard resupply — 1,500 of 1,500 pieces on time, new cash SGD 1,200.00, net contribution SGD 28,450.00 (recommended); expedited resupply — same delivery, SGD 1,587.50 / SGD 28,062.50; immediate transfer — same delivery, SGD 2,700.00 / SGD 26,950.00; a smaller quantity of 1,200 pieces — no new cash, SGD 26,700.00 in V3 (SGD 26,506.20 in V7, where the option also proposed a later due date). Keeping current conditions or adding overtime alone was blocked by the material gap (250 each of inner ring, outer ring, ball set and cage; 500 seals).

Observed issues. In V7, a later model reply called the resupply "arrived 09-27 10:45" while the factory clock showed 08:33; the card, the execution log and the reply written from the recorded result all said "arrives". In V8, after the SO-003 execution the model re-ran the schedule and posted a redundant zero-change plan card marked ready for approval, again describing the resupply as in place before it had arrived; and a free-text answer about one SO-002 option put its net-contribution difference to the recommended option at SGD 152 instead of SGD 1,521.49. These issues are recorded, not fixed. Option-card metrics are computed by deterministic services; free-form model explanations can still misstate numbers or timing. Approval and execution use the structured option and recorded scope, so users should verify a recommendation against the card and the execution record.

Not verified. A pilot in a real plant; any baseline of the manual process; load with many users or conversations; a public deployment; a real ERP, MES or purchasing connection.

10. Limits and pilot

- **Search is time-limited.** A feasible schedule is not proven optimal, and finding none in time is not proof that none exists; the interface and the Agent say which case applies.
- **Economics are catalog assumptions.** Prices, premiums and deductions are declared synthetic values; net contribution is not accounting profit, and the adverse case is not a probability.

- **The factory is simulated.** The connector is tested with example payloads only; no real purchase order or customer contract is created.
- **One reference factory.** Results come from one synthetic plant and single runs on one laptop.
- **Model wording.** Free-form model explanations can still misstate numbers or timing (section 9). Option-card metrics come from deterministic services, and approval and execution use the structured option and recorded scope; users should verify a recommendation against the card and the execution record.
- **Capacity.** Option comparisons run in parallel, but multi-user capacity and many simultaneous disruptions have not been measured.
- **Access.** Two demonstration roles without passwords; a pilot needs real sign-in and HTTPS.

Pilot (four to six weeks, one discrete-assembly plant). Weeks 1–2: map and validate the plant's data through the connector and record the manual baseline. Then run the Agent in shadow mode beside the manager, and finally let it prepare decisions that the manager approves, with purchase orders still issued through the plant's own process. A plant adapter would replace the simulator behind the same HTTP contract. Measured against the baseline: decision lead time; on-time production completion against the original committed date, with agreed revised dates and the number and length of date changes tracked separately; expediting and overtime spend per disruption; approved responses executed without rework; and manager time per disruption. Forecast completion, actual production and shipment are kept apart.